

TUF+ Arrays Part-IV Largest subarray with K sum

Approach:

- Maintain a running sum/prefix sum when traversing
- Initialize maxi to store the maximum length of the subarray
- Create hashmap to store the sums
- If at any index the $\text{sum} == k$, then subarray from start to that index has $\text{sum} = k$, update length as $i+1$
- If $\text{sum} - k$ in hashmap at index j , then subarray $(j+1, \dots, i)$ has $\text{sum} = k$
- If not found, update the value and insert into hashmap
- Return maxi

TUF+ Arrays Part-IV Count number of subarrays with given XOR k

Approach:

- Create running XOR/prefix XOR that acts like prefix sums but performs bitwise XOR
- If $\text{prefixXOR} \oplus k$ has appeared before, then there exists a subarray ending at the current index with $\text{XOR} = k$.
- Use a hashmap (freq) to store counts of prefix XOR values seen so far.
- Iterate through the map, update map and return final count

LC:3-Longest Substring Without Repeating Characters

Approach:

- Use a **sliding window** with two pointers (l and j) to track current substring.
- Maintain a **set** (1as) to store unique characters in the current window.
- If $s[i]$ not in set $\rightarrow$ add it, update maxlen.
- If $s[i]$ already in set $\rightarrow$ shrink window from left j
- Update and return maxlen